



Rapport de Travaux Pratiques

Timer ATmega2560 avec PWM et conversion ADC Environnement : Atmel Studio

Professeur : Mr BARRO

Réalisé par : Payié Mariane ELOLA

Sephora Maurane BAYALA

Mamadou LATIF SALIFOU HAMANI

Année Académique : 2025-2026

SOMMAIRE

1.	INTRODUCTION.....	3
2.	Matériel et environnement	3
2.1.	Composants matériels	3
2.2.	Affectation des ports	3
2.3.	Environnement logiciel	3
3.	Montage.....	4
4.	Logigramme du programme	5
5.	Conversion analogique-numérique (ADC).....	5
5.1.	Rôle de la fonction	5
5.2.	Explication des registres utilisés.....	6
6.	Génération du signal PWM avec le Timer 1	6
6.1.	Principe du PWM.....	6
6.2.	Mode Fast PWM 10bits (mode 7).....	6
6.3.	Calcul de la fréquence (OCR1A).....	6
6.4.	Calcul du rapport cyclique (OCR1B)	7
7.	Programme principal (main)	7
7.1.	Initialisation des sorties.....	7
7.2.	Boucle principale	7
8.	Conclusion.....	7

1. INTRODUCTION

Ce travail pratique porte sur la programmation du microcontrôleur ATmega2560. L'objectif est de générer un signal PWM (modulation de largeur d'impulsion) dont le rapport cyclique varie automatiquement en fonction d'une tension lue sur une entrée analogique. Concrètement, on tourne un potentiomètre et la largeur du signal change en temps réel.

Le programme utilise deux fonctionnalités principales du microcontrôleur :

- Le convertisseur analogique-numérique (ADC) pour lire la tension sur une broche,
- Le Timer 1 en mode Fast PWM pour générer le signal de sortie sur la broche OC1B.

2. Matériel et environnement

2.1. Composants matériels

Composant	Référence / Valeur	Rôle
Microcontrôleur	ATmega2560	Unité centrale de traitement
Potentiomètre	10 kΩ	Réglage de la tension
LED jaune	5 mm	Indicateur visuel sur PB6
Résistance	220 Ω	Limitation du courant dans la LED
Alimentation	5 V	Alimentation du système
Fils de connexion	Jumper wires	Connexions entre composants
Breadboard	830 points	Support de câblage sans soudure

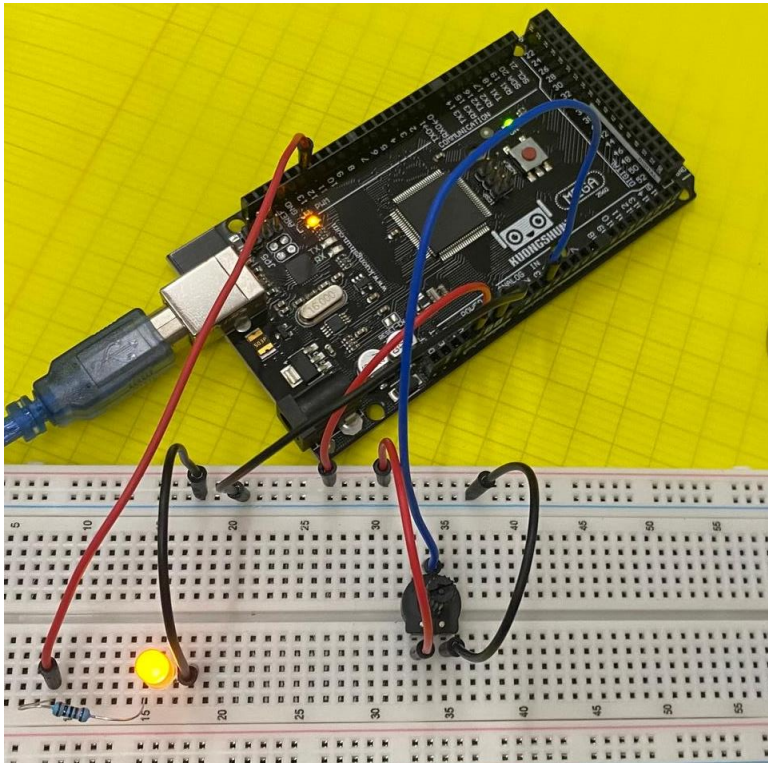
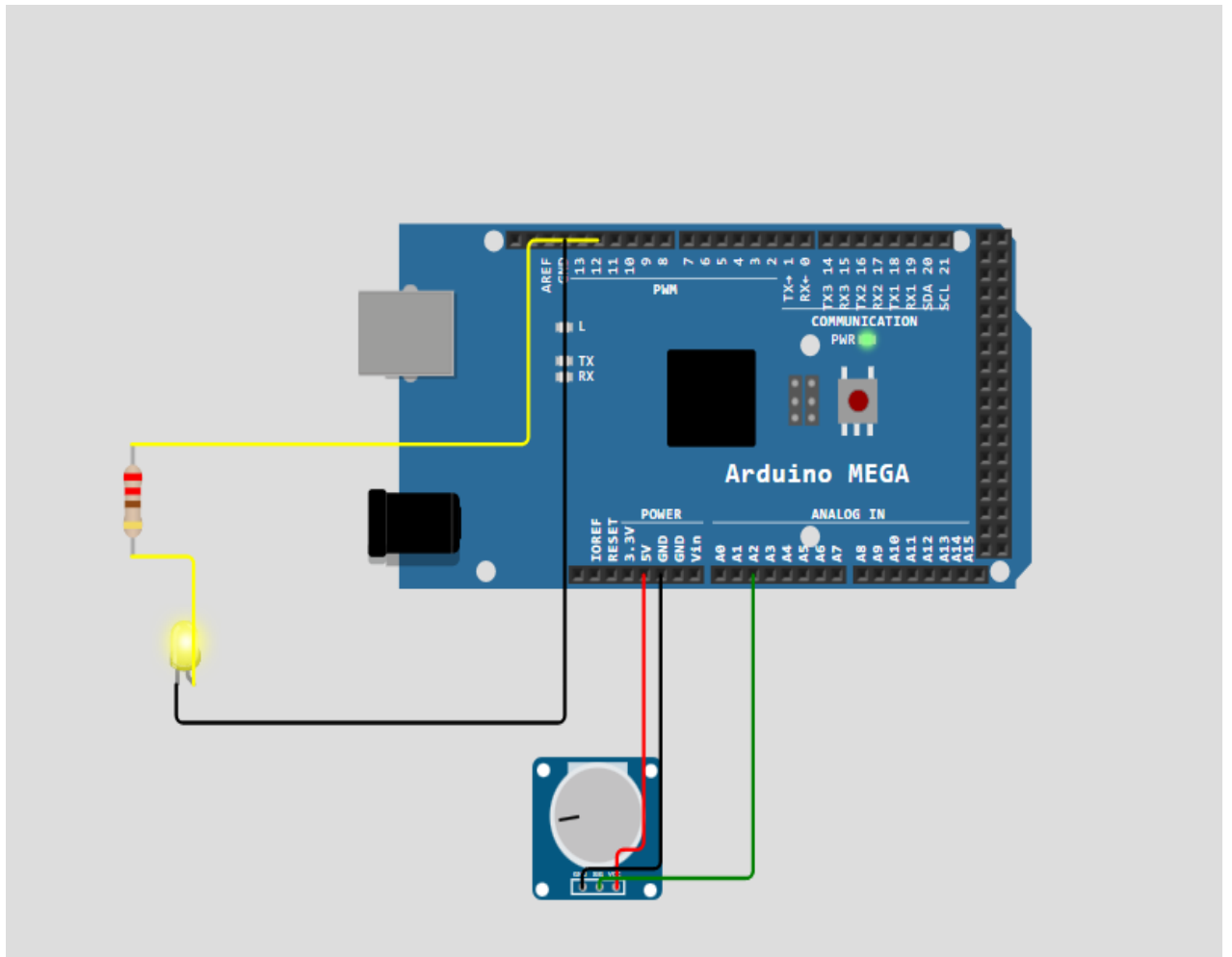
2.2. Affectation des ports

Port	Broche(s)	Fonction
PORT B	PB6 (OC1B)	Sortie signal PWM — broche led_jaune
PORT F	PF2 (ADC2)	Entrée analogique — potentiomètre

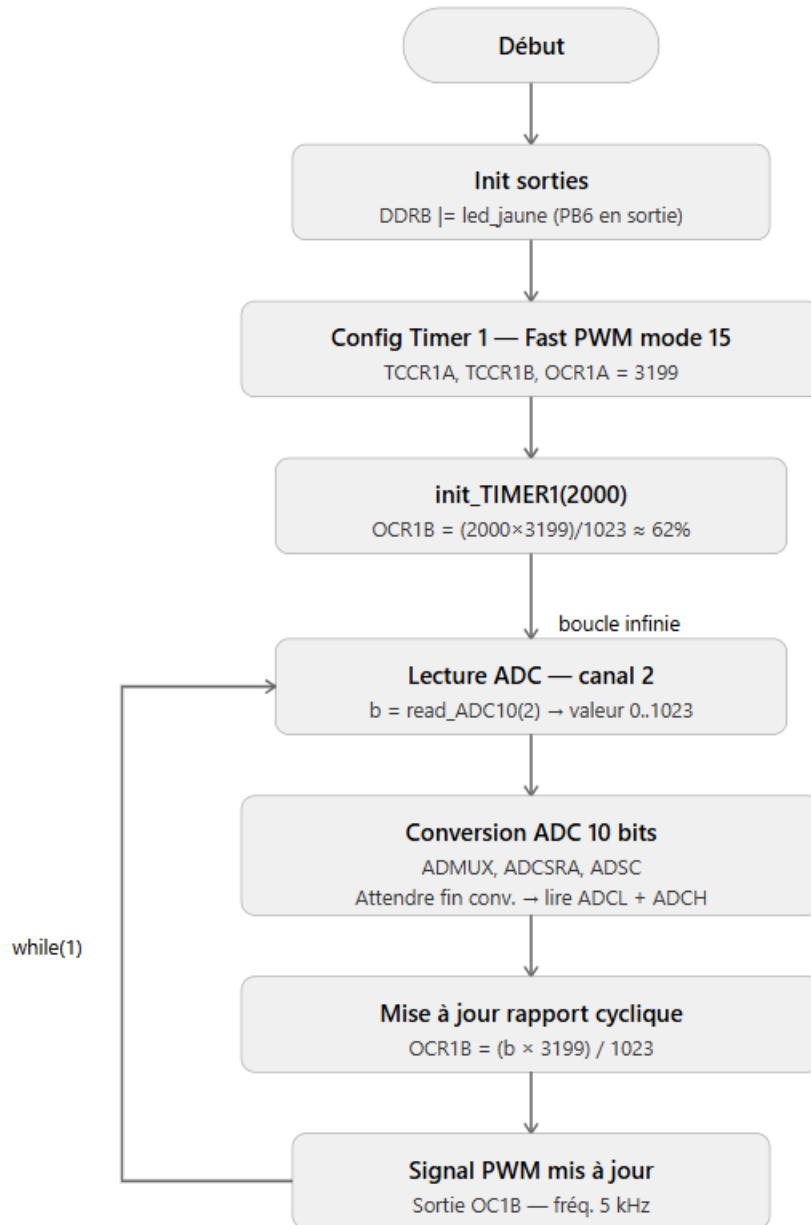
2.3. Environnement logiciel

- **Langage** : C standard (C99), compilé avec avr-gcc
- **IDE** : Atmel Studio 7.0 (AVR Studio)
- **Bibliothèques** : <avr/io.h>, <util/delay.h>
- **Fréquence CPU** : F_CPU = 16 MHz

3. Montage



4. Logigramme du programme



5. Conversion analogique-numérique (ADC)

5.1. Rôle de la fonction

La fonction `read_ADC10(unsigned char input_channel)` lit une tension analogique sur l'une des entrées du microcontrôleur et renvoie une valeur numérique entre 0 et 1023 (conversion sur 10 bits). Plus la tension est grande, plus la valeur renvoyée est proche de 1023.

5.2. Explication des registres utilisés

Le microcontrôleur dispose de 16 entrées analogiques (canaux 0 à 15). Selon le canal choisi, la configuration diffère légèrement :

Canaux 0 à 7 (ADMUX et DIDR0) : Pour les 8 premiers canaux, on configure le registre **ADMUX** en activant le bit **REFS0** pour choisir la tension de référence AVCC (5 V). On y ajoute le numéro du canal directement. On désactive aussi l'entrée numérique correspondante via **DIDR0** pour réduire la consommation.

```
ADMUX = (1 << REFS0) + input_channel; DIDR0
= (1 << input_channel);
```

Canaux 8 à 15 (ADMUX, ADCSRB et DIDR2) : Pour les canaux supérieurs, il faut en plus activer le bit **MUX5** dans le registre **ADCSRB**. C'est un bit supplémentaire qui permet d'atteindre les canaux au-delà du canal 7. On passe **input_channel - 8** à **ADMUX** pour revenir dans la plage 0–7.

```
ADMUX = (1 << REFS0) + (input_channel - 8);
ADCSRB = (1 << MUX5);
DIDR2 = (1 << (input_channel - 8));
```

Démarrage de la conversion (ADCSRA) : Le registre **ADCSRA** configure et lance la conversion. On active le convertisseur avec **ADEN**, on règle la vitesse d'horloge du convertisseur à $F_{CPU}/128$ (soit 125 kHz, valeur recommandée pour une bonne précision) avec les bits **ADPS2:0** tous à 1, et on démarre la conversion avec **ADSC**.

```
ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0);
ADCSRA |= (1 << ADSC);
while (ADCSRA & (1 << ADSC)); // attendre la fin de conversion return
(ADCL + (ADCH << 8)); // résultat sur 10 bits
```

On attend la fin de la conversion en surveillant le bit **ADSC** qui repasse à 0 automatiquement. Le résultat est ensuite lu dans les registres **ADCL** (8 bits de poids faible) et **ADCH** (2 bits de poids fort), qu'on combine pour obtenir la valeur finale sur 10 bits.

6. Génération du signal PWM avec le Timer 1

6.1. Principe du PWM

Un signal PWM est un signal carré dont la fréquence est fixe mais dont le rapport cyclique (la proportion de temps à l'état haut) varie. Plus le rapport cyclique est élevé, plus la tension moyenne vue par la charge est élevée. C'est utile pour commander la vitesse d'un moteur ou la luminosité d'une LED par exemple.

6.2. Mode Fast PWM 10bits (mode 7)

On utilise le Timer 1 en mode Fast PWM avec **OCR1A** comme valeur maximale (TOP). Les bits de mode **WGM13:0** sont configurés à la valeur 15 (= 1111 en binaire), ce qui active ce mode précis.

```
TCCR1A = (1 << COM1B1) | (1 << WGM11) | (1 << WGM10);
TCCR1B = (1 << CS10) | (1 << WGM13) | (1 << WGM12);
```

Le bit **COM1B1** active la sortie automatique sur la broche OC1B : le signal passe à 1 en début de période et retombe à 0 quand le compteur atteint **OCR1B**. Le bit **CS10** signifie qu'on n'utilise pas de prédiviseur ($N = 1$), donc le timer tourne à la même fréquence que le processeur (16 MHz).

6.3. Calcul de la fréquence (OCR1A)

La fréquence du signal PWM est déterminée par la valeur chargée dans **OCR1A** selon la formule :

$$f_{oc1a} = f_{clk} / (N \times (OCR1A + 1))$$

On souhaite une fréquence de 5 kHz, avec $f_{clk} = 16$ MHz et $N = 1$:

$$OCR1A = (16\ 000\ 000 / (1 \times 5\ 000)) - 1 = 3199$$

On charge donc la valeur 3199 dans **OCR1A**. Le timer comptera de 0 à 3199 (soit 3200 pas) avant de recommencer, ce qui donne bien une période de 200 µs et donc 5 kHz.

6.4. Calcul du rapport cyclique (OCR1B)

Le rapport cyclique dépend de la valeur dans **OCR1B** par rapport à **OCR1A** :

$$\text{Rapport cyclique} = (OCR1B + 1) / (OCR1A + 1)$$

Dans la boucle principale, la valeur lue par l'ADC (a, entre 0 et 1023) est convertie en valeur pour **OCR1B** :

```
OCR1B = (int)((a * 3199) / 1023);
```

Cette formule fait une règle de trois simple : si l'ADC vaut 0, OCR1B vaut 0 (rapport cyclique nul). Si l'ADC vaut 1023 (tension max), OCR1B vaut environ 3199 (rapport cyclique de 100 %). Ainsi la largeur du signal suit exactement la position du potentiomètre.

7. Programme principal (main)

7.1. Initialisation des sorties

On déclare la broche PB6 (définie comme **led_jaune = 0x40**) en sortie, et on l'éteint au démarrage :

```
#define led_jaune 0x40 // PB6
DDRB |= led_jaune;    // PB6 en sortie
PORTB &= ~led_jaune;  // LED éteinte
```

Le masque binaire 0x40 = 0100 0000 correspond au bit 6 du port B, c'est-à-dire la broche PB6. On utilise l'opérateur **|=** pour ne modifier que ce bit sans affecter les autres broches.

7.2. Boucle principale

Le timer est initialisé une première fois avec une valeur de démarrage de 2000 (environ 62 % de rapport cyclique), puis la boucle infinie met à jour en continu le rapport cyclique selon la lecture ADC :

```
init_TIMER1(2000); // démarrage avec rapport cyclique ~62% while
(1) {
    b = read_ADC10(2); // lire le canal 2 (broche ADC2)
    init_TIMER1(b);    // mettre à jour OCR1B
}
```

À chaque tour de boucle, on relit la valeur analogique sur le canal 2 et on reconfigure le timer en conséquence. Le signal PWM sur OC1B change donc en quasi temps-réel selon la tension présente sur l'entrée ADC2.

8. Conclusion

Ce programme illustre deux mécanismes fondamentaux de la programmation de microcontrôleurs :

- La conversion analogique-numérique (ADC) permet de transformer une information physique continue (une tension) en une valeur numérique exploitable par le programme.
- Le Timer en mode Fast PWM permet de générer un signal périodique à fréquence fixe dont on peut contrôler précisément la durée à l'état haut.
- La combinaison des deux permet de réaliser un asservissement simple : la position d'un potentiomètre pilote directement le rapport cyclique du signal de sortie.

Le raisonnement suivi repose sur la compréhension des formules qui lient les valeurs des registres aux caractéristiques électriques du signal. Chaque valeur chargée dans un registre a une signification physique précise : fréquence, rapport cyclique, canal d'entrée, tension de référence. C'est cette logique registre-par-registre qui est au cœur de la programmation embarquée sur AVR.